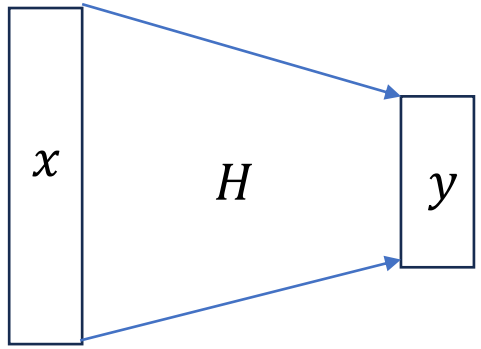
COS 433: Cryptography

Announcements:

- Problem Set 4 due Wed 11/11

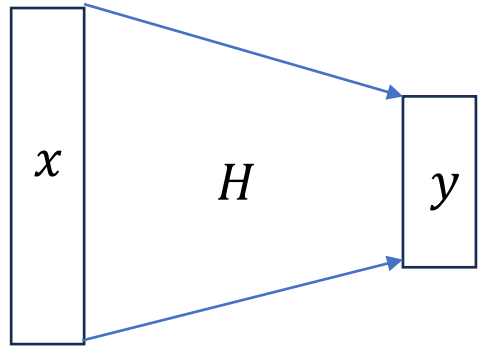
Last time:

1. CRHFs allow for signing long messages with short keys
2. CHRFs based on the discrete log assumption
3. Length extension of CRHFs (Merkle trees)
4. Random Oracle heuristic



Random Oracle Heuristic

1. Design a crypto scheme that makes black-box use of a hash function H.



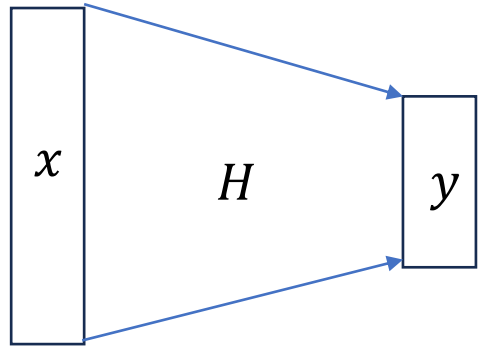
2. Prove security "in the random oracle model:"
   - H is a uniformly random function
   - Adversary is of the form $A^{H(\cdot)}$, making black-box calls to H.

3. Heuristically instantiate with a concrete hash function family $\{H_k\}$

4. Philosophy (unproven): carefully designed hash functions should not have non-black-box attacks.

Random Oracle Heuristic

1. Design a crypto scheme that makes black-box use of a hash function H.



2. Prove security "in the random oracle model:"
   - H is a uniformly random function
   - Adversary is of the form $A^{H(\cdot)}$, making black-box calls to H.

3. Heuristically instantiate with a concrete hash function family $\{H_k\}$

Extremely common/powerful tool for designing practical cryptography

But: sometimes unsound, need to be careful

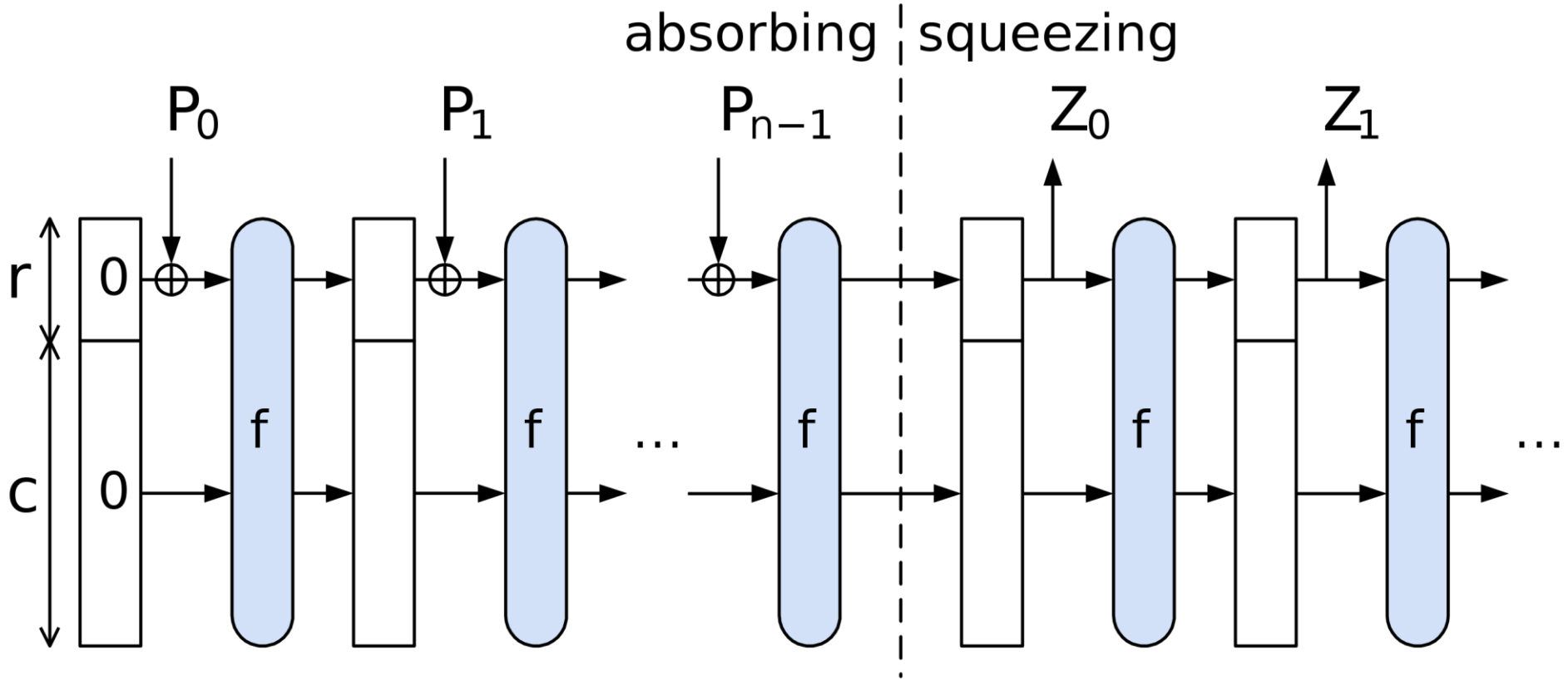
Random Oracle Instantiations (heuristic)

- SHA-2 (2001-)
  - Output length: 256 or 512 bits
  - Fixed-length hash function; use Merkle-Damgard to extend input length
    - This works for collision resistance but not every security property!
    - Need to worry about length extension attacks

- SHA-3 (2015-)
  - Output length: 256 or 512 bits
  - “Sponge construction”
  - Unbounded input length!

- Theoretical: “obfuscated pseudorandom function” (we won’t discuss)

SHA-3: the Sponge Construction



- f: public, AES-like permutation (composition of several simple permutations for 24 rounds)
  - Unlike AES: keyless!
- Proceeds block-by-block
  - Repeatedly: XOR in a block of your input, then apply f
  - Repeatedly: output a block of the current memory contents, then apply f

Applications of Random Oracles

- Two practical signature schemes: RSA and Schnorr/DSA

- Practical, CCA-secure public-key encryption (we won’t do this in class)

RSA Signatures

“Textbook” RSA Signatures

$\text{sk} = (N, d)$
$\text{vk} = (N, e)$



$m \in \mathbb{Z}_N^\times$



(N, e)

$(m, \sigma = m^d \bmod N)$



$\text{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv m \pmod N$

Notes:

- Restricted to $m \in \mathbb{Z}_N^\times$
- Secure if the queries and challenge m^* are all random and outside the adversary’s control.

RSA Signatures: “Full Domain Hash”

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \{0,1\}^*$



(N, e)

$(m, \sigma = \boldsymbol{H}(m)^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv \boldsymbol{H}(m)$

Notes:

- Uses a hash function $\boldsymbol{H}: \{0,1\}^* \to \mathbb{Z}_N$ (turns out: bounded domain hash function ok)
- This is just hash-and-sign applied to textbook RSA!
- But textbook RSA is not secure to begin with, so why is this secure?

RSA Signatures: "Full Domain Hash"

$\mathrm{sk} = (N, d)$
$\mathrm{vk} = (N, e)$



$m \in \{0,1\}^*$



(N, e)

$(m, \sigma = \boldsymbol{H}(m)^d \bmod N)$



$\mathrm{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv \boldsymbol{H}(m)$

Existential Unforgeability: for all efficient adversaries $A^{\mathrm{Sign}(\mathrm{sk},\cdot)}$



vk

m

$\mathrm{Sign}(\mathrm{sk}, m)$



$$\Pr_{(\mathrm{sk},\mathrm{vk})}\left[A^{\mathrm{Sign}(\mathrm{sk},\cdot)}(\mathrm{vk}) \to (m^*, \sigma^*) : m^* \text{ not queried and } \mathrm{Verify}(\mathrm{vk}, m^*, \sigma^*) = 1\right] = \mathrm{negl}(\lambda)$$

RSA Signatures: "Full Domain Hash"

$\text{sk} = (N, d)$
$\text{vk} = (N, e)$





(N, e)

$(m, \sigma = \boldsymbol{H}(m)^d \bmod N)$



$m \in \{0,1\}^*$

$\text{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv \boldsymbol{H}(m)$

0-time security: for all efficient adversaries A



vk

m^*, σ^*



$$\Pr_{(\text{sk},\text{vk})}[A(\text{vk}) \to (m^*, \sigma^*): \text{Verify}(\text{vk}, m^*, \sigma^*) = 1] = \text{negl}(\lambda)$$

RSA Signatures: "Full Domain Hash"

$\text{sk} = (N, d)$
$\text{vk} = (N, e)$



$m \in \{0,1\}^*$



(N, e)

$(m, \sigma = \boldsymbol{H}(m)^d \bmod N)$



$\text{Verify}((N, e), m, \sigma) = 1$ if $\sigma^e \equiv \boldsymbol{H}(m)$

0-time security (random oracle model): for all efficient adversaries $A^{H(\cdot)}$



(N, e)

m

$H(m)$

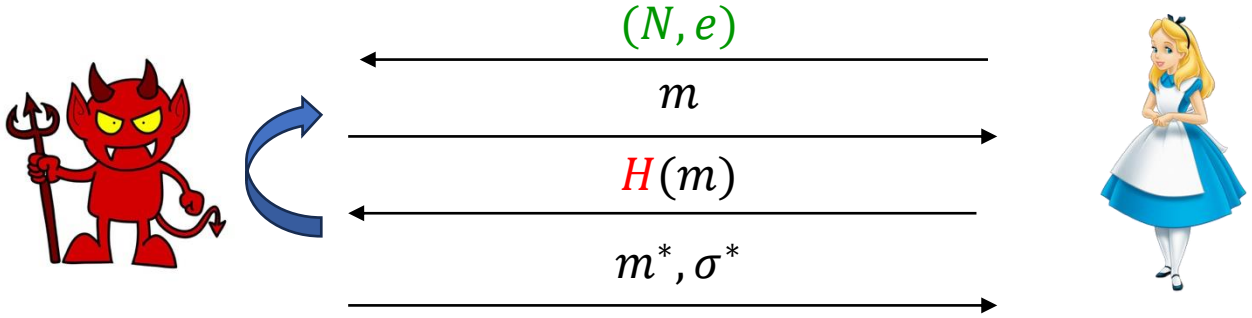
m^*, σ^*



$$\Pr_{p,q,e}[A^H(N, e) \to (m^*, \sigma^*): (\sigma^*)^e \equiv H(m^*)] = \text{negl}(\lambda)$$

RSA Signatures: “Full Domain Hash”

0-time security (random oracle model): for all efficient adversaries $A^{H(\cdot)}$



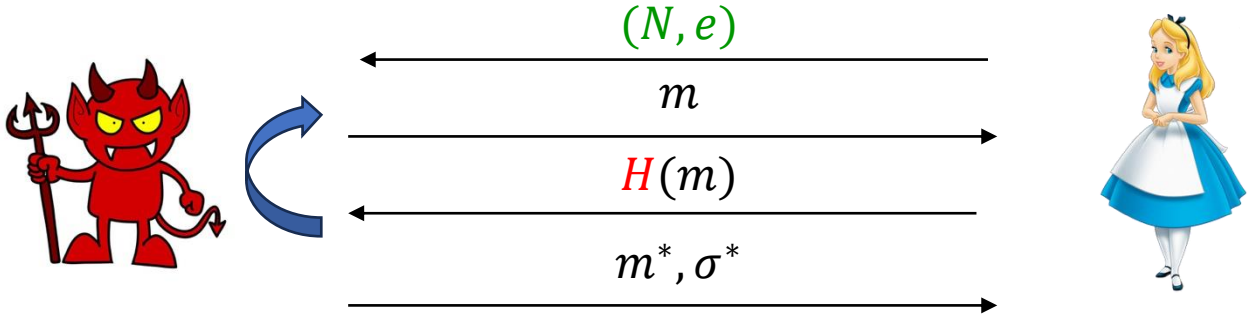
$$\Pr_{p,q,e}[A^H(N,e) \to (m^*, \sigma^*) : (\sigma^*)^e \equiv H(m^*)] = \mathrm{negl}(\lambda)$$

Proof (from hardness of RSA):

- We may assume  queries $H(m^*)$
- We can represent H by a poly-length list of pairs (m, y)
- We can break RSA by guessing which query adds m^* to the list.

RSA Signatures: “Full Domain Hash”

0-time security (random oracle model): for all efficient adversaries $A^{H(\cdot)}$



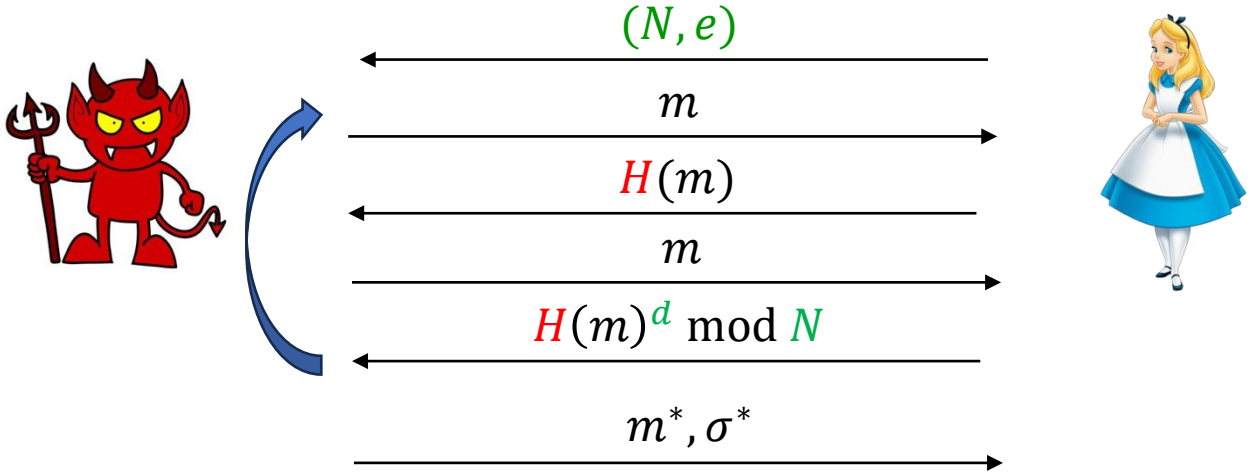
$$\Pr_{p,q,e}[A^{H}(N,e) \to (m^*, \sigma^*): (\sigma^*)^e \equiv H(m^*)] = \mathrm{negl}(\lambda)$$

Proof (from hardness of RSA):

- Given $(N, e, y = x^e)$: guess index i
- Run  , set $y_i = y$
- If  wins with probability ϵ, then  outputs x with probability $\frac{\epsilon}{T}$

RSA Signatures: "Full Domain Hash"

Full security (random oracle model): for all efficient adversaries $A^{H(\cdot),\mathrm{Sign}(\mathrm{sk},\cdot)}$



$$\Pr_{p,q,e}\left[m^* \text{ not queried to } \mathrm{Sign}(\mathrm{sk},\cdot) \text{ and } (\sigma^*)^e \equiv H(m^*)\right] = \mathrm{negl}(\lambda)$$

Proof idea (from hardness of RSA): signing queries are useless

- Combine the previous proof with the proof of many-time random message security
- Given $(N, e, y = x^e)$: guess index i
- On each m_j query to H (for $j \neq i$) sample $y_j = x_j^e$. Set $y_i = y$.
- Can answer all signing queries with corresponding x_j

Schnorr/DSA Signatures

Signature scheme defined over:

- Group G of size p
  - In practice: “elliptic curve group” (ECDSA)
  - Given generator g for G
- Scheme is secure in the ROM, assuming the hardness of discrete log
- DSA slightly different from Schnorr, most likely due to patent issues (we will ignore the distinction)

Schnorr Signatures

$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$
$\mathrm{vk} = g^x \in G$





$y = g^x$



$m \in \{0,1\}^*$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x
  - (For RSA, we used d to invert eth powers)

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$
$\text{vk} = g^x \in G$





$y = g^x$

(m, x)



$m \in \{0,1\}^*$

$\text{Verify}(y, m, \sigma) = 1$ if $g^\sigma = y$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x
- Trivial 0-time secure scheme: send x
  - Clearly 1-time insecure

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$
$\text{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = x + r)$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y \cdot R$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x <u>but does not reveal</u> x
- Another attempt: send x **masked** by a random value r (call this z)
  - Equation: $g^z = y \cdot g^r$
  - Send $R = g^r$ as part of the signature
  - Good news: signature seems not to reveal x

Schnorr Signatures

$\mathrm{sk} = x \leftarrow \mathbb{Z}_p$
$\mathrm{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = x + r)$



$\mathrm{Verify}(y, m, R, z) = 1$ if $g^z = y \cdot R$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x but does not reveal x
- Another attempt: send x **masked** by a random value r
  - Bad news: 0-time Broken!
  - Pick any z, set $R = g^z \cdot y^{-1}$

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$
$\text{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = H(R,m) \cdot x + r)$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

How should we sign messages?

- Idea: we need to do something that requires knowledge of x <u>but does not reveal</u> x
- Actual scheme: uses a hash function H to determine how to mask x
  - Set $R = g^r$
  - Compute $s = H(R,m)$
  - Set $\sigma = (R, s \cdot x + r)$

Schnorr Signatures

$\text{sk} = x \leftarrow \mathbb{Z}_p$

$\text{vk} = g^x \in G$



$m \in \{0,1\}^*$



$y = g^x$

$(m, R = g^r, z = H(R, m) \cdot x + r)$



$\text{Verify}(y, m, R, z) = 1$ if $g^z = y^{H(R,m)} \cdot R$

Why is this secure?